SOFTWARE ENGINEER · REXBURG, IDAHO · BYU-IDAHO ALUMNUS

Chase Moses · chsmoses@outlook.com · github.com/chasemoses · linkedin.com/in/chase-moses

# I build *.NET* applications, with *AI agents* on the team.

My day runs through a dashboard I built where AI agents plan, build, and review work, and I approve every plan and merge every change. Below is that process, and the projects that show it: work systems rebuilt so the code can be public, a client app, and the tools I use every day.

| 8 | 7 | 71 | C# · Blazor · SQL Server |
|---|---|---|---|
| AI agents in my daily workflow | projects with design notes | automated tests, all passing | what I work in every day |

HOW I WORK WITH AI

## I run AI agents the way a good team runs development.

Clear tasks, approved plans, independent review, and a person who decides what ships. I run this loop every day on a platform I built, and it's how I think about AI at work too: the speed is real, and so is the need to check what comes back.

I build with Claude Code, OpenAI's Codex and ChatGPT, and GitHub Copilot, and I run local models on my own server when data shouldn't leave the network.

01
### Capture
A task lands on the board, from me or from my assistant agent in chat. Ideas I only want to remember wait in an inbox until I act on them.

02
### Plan
The builder agent writes a plan with acceptance criteria.

◆ I approve the plan before any work starts.

03
### Build
Work happens on a branch in a separate worktree and comes back as a pull request with test evidence and a plain-language "How it works."

04
### Review
A separate reviewer agent checks the evidence against every acceptance criterion. It can't fix the work itself, and two failed reviews escalate to me.

05
### Ship
Merging deploys automatically through a self-hosted runner, and rolls back if the new version isn't healthy.

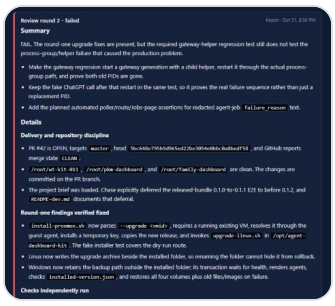
◆ I merge. Nothing ships without me.

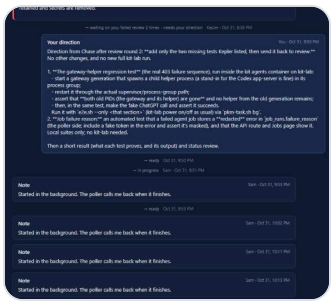
06
### Improve
When the loop fails, I fix the loop, not just the output: a new rule in the agents' instructions and a test that keeps it there.

A REAL EXAMPLE

### One task, start to finish

Upgrading the agent kit to 0.1.1: the reviewer failed the pull request twice, the second time because a regression test didn't reproduce the real production failure. The task escalated to me, I narrowed the fix to the two missing tests, and it passed the next review.





## Loop engineering: when the loop fails, I fix the loop

Agents make mistakes the same way people do, so a better prompt is rarely enough. Each of these came from the dashboard's real history and became a rule, enforced by code or instructions, with a test behind it.

**WHAT HAPPENED**

An agent edited a live checkout instead of opening a pull request, and the reviewer passed it because the content was right.

**WHAT CHANGED**

Repo changes must arrive as a pull request from a separate worktree. The reviewer now fails any work without one.

**WHAT HAPPENED**

Scheduled flight-price checks failed silently for seven weeks after their routes changed, and every run still looked successful.

**WHAT CHANGED**

One job checks every active route, and agents report failed runs, so a failure shows red on the Jobs page with its reason.

**WHAT HAPPENED**

A task was marked blocked with the real reason buried at the bottom of a long result document.

**WHAT CHANGED**

The API refuses to block a task without a reason, and the reason appears right in the activity log.

**WHAT HAPPENED**

Hitting a usage limit burned through all three retries in a minute and blocked the task.

**WHAT CHANGED**

The poller reads the provider's reset time, waits it out, and resumes the work instead of failing it.

## Where it shows up

### At work

GitHub Copilot every day. For larger changes I write a design proposal with an AI assistant and review it before any code is written. An AI-assisted investigation found the bottleneck behind my filtered-index fix, and I started writing guidance for our developers on where AI helps and where its output needs checking.

### Phased planning

Home Management System was built in research, plan, check, execute, and verify phases with specialized agents. Every plan and verification report is kept in the repository.

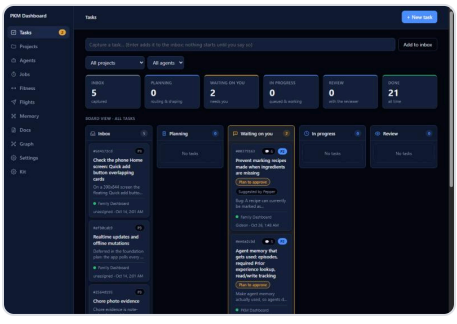
### Cost and reliability

Routine jobs run as plain scripts with no model calls, and an idle system costs nothing. When an agent setup quietly died after hitting a usage cap, I rebuilt its job as a lightweight script so it couldn't happen again.

### No lock-in

The dashboard knows nothing about any single AI tool. Agents running in OpenClaw, Claude Code, or Codex connect through the same API, so changing providers doesn't mean rebuilding.

## Selected work



**PERSONAL PROJECT** 8 AI agents on the board

### PKM Agent Dashboard

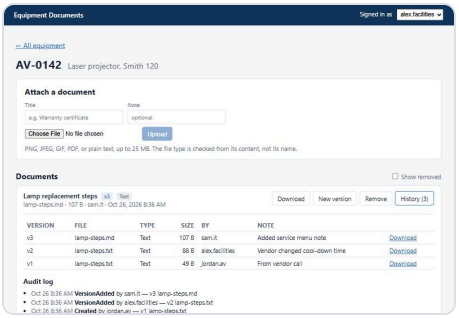
What I use every day · self-hosted on my home lab · 256 commits, 43 pull requests

The control plane for my AI agents: a task board, shared memory and docs, scheduled jobs, and a live graph of everything they're doing. Agents plan, build, and review work through it; I approve every plan and merge every change.

- A separate reviewer agent checks evidence against every acceptance criterion and escalates to me after two failures
- A plain-code poller dispatches tasks and scheduled jobs, so an idle system costs no model usage
- Packaged as an installable kit for Windows, Mac, Linux, and Proxmox, with backups and rollback-safe upgrades

AI agents · Agent orchestration · MCP · React · Node.js · Express · PostgreSQL · Docker · GitHub Actions · Proxmox

*Private repository*




**WORK, REBUILT** **23** automated tests

## Versioned Document Attachments

Rebuilt from a document management system I designed and built at FileOnQ

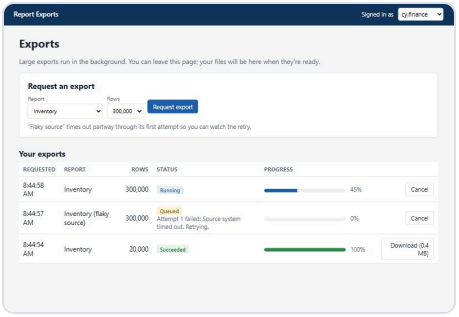
Attach warranties, manuals, and photos to campus equipment, with full version history, safe content checks, and an audit trail of every change.

- Uploads are checked by their actual content, so a renamed .exe is rejected
- Two people uploading at the same moment can't overwrite each other
- Removing a document is a soft delete with a required reason, and can be undone

C# · Blazor · Entity Framework Core · SQL Server · ASP.NET Core · Secure file handling · Concurrency · xUnit

**Read the code and design notes** https://github.com/chasemoses/VersionedDocuments

---




**WORK, REBUILT** **18** automated tests

## Background Report Exports

Rebuilt from background download processing I built at FileOnQ

Large exports move out of the web request. The page responds immediately while a hosted worker builds the file and reports live progress.

- The queue lives in SQL Server, so a restart mid-export resumes instead of losing work
- Temporary failures retry with backoff, and users can cancel at any point
- Only the person who requested a file can download it

C# · ASP.NET Core · Hosted services · Blazor · Entity Framework Core · SQL Server · Concurrency · Authorization · xUnit

**Read the code and design notes** https://github.com/chasemoses/BackgroundExports

---




**WORK, REBUILT** **−97%** index entries

## Filtered Indexes for a Slow Import

Reproduces a performance fix I made at FileOnQ

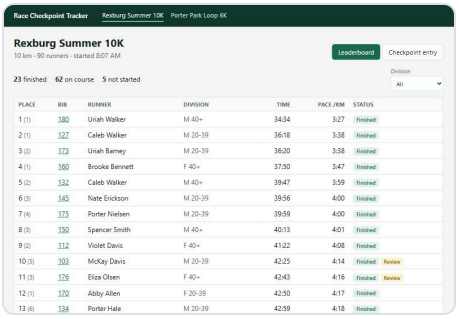
A bulk import was slow because SQL Server maintained seven index entries for every row, mostly for empty values. Filtered indexes fixed it.

- Index entries per inserted row fell from 7.00 to 0.20; import time fell 77%
- Actual execution plans confirm searches still use the indexes
- The cases that get worse are measured too, so the trade-off is a decision

SQL Server · Indexing · Execution plans · Performance · C# · ADO.NET · Technical writing

**Read the code and design notes** https://github.com/chasemoses/EavFilteredIndexes

---




**WORK, REBUILT** **24** automated tests

## Race Checkpoint Tracker

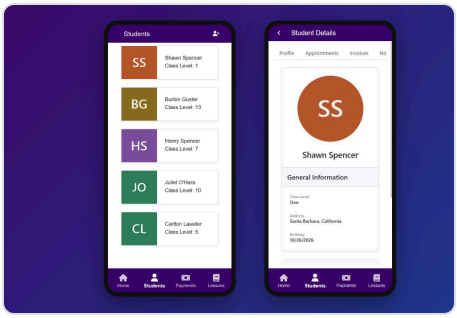
Rebuilt from a Python app I wrote for the City of Rexburg

Volunteers type bib numbers at checkpoints, and the app turns those entries into live results for point-to-point and multi-lap courses.

- Works out which lap each crossing belongs to and ignores double taps
- Flags finishers who missed a checkpoint for review instead of disqualifying them
- Browser testing caught a time-zone bug every unit test missed

Angular · TypeScript · RxJS · ASP.NET Core · REST APIs · Entity Framework Core · SQL Server · xUnit

**Read the code and design notes** https://github.com/chasemoses/RaceCheckpointTracker




**CLIENT PROJECT** **3** platforms, one codebase

## Wisdom in Learning

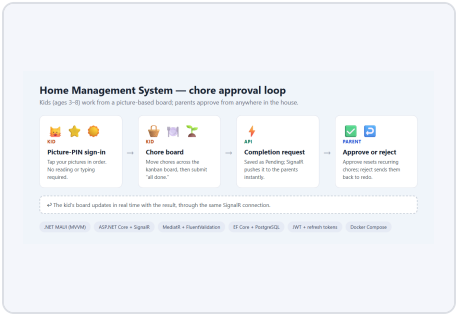
Sole developer for a company teaching students in China · built 2023, rewritten 2024

Student management on Windows, Android, and iOS: students and families, class levels, payments, and invoice history, behind a REST API.

- The 2024 rewrite moved the UI into shared Blazor components hosted in .NET MAUI
- One codebase gives a desktop sidebar and a mobile tab bar
- The README is candid about what I'd do differently today

.NET MAUI · Blazor · ASP.NET Core · REST APIs · Entity Framework Core · SQL Server · Clean Architecture · MediatR

**Read the code and design notes** https://github.com/chasemoses/WisdomInLearningCopy




**PERSONAL PROJECT** **74** commits of phased work

## Home Management System

A self-hosted chore app for my family · in active development

Kids ages 3–8 sign in by tapping pictures and work from a picture-based board; parents approve finished chores from their phones in real time.

- Planned and built in verified phases with AI assistance; every plan is kept with the code
- A feasibility spike deliberately said no to fingerprint sign-in for kids
- Notifications survive a phone in a pocket with a custom reconnect policy

AI agents · .NET MAUI · ASP.NET Core · SignalR · Entity Framework Core · PostgreSQL · Docker · JWT · MediatR

*Private repository*

# Background

### NOW

Software Developer at FileOnQ since 2024: full-stack C#/.NET, Blazor, Entity Framework Core, and SQL Server, in two-week sprints.

### BEFORE

IT Support Technician at the City of Rexburg; Tier 2 technical support at M9 Creations.

### EDUCATION

B.S. Computer Software Engineering, Brigham Young University–Idaho, 2023.

### AI TOOLS

GitHub Copilot daily at work; Claude Code for these projects. Each README notes how AI was used and what I checked myself.